

Notas de mi Procedimiento

AUTOR: Oriol Farrés

Día 1: 20 Octubre 2025 (8h)

Primera toma de contacto. Explicaciones, cursos Anthropic, huella...

Día 2: 21 Octubre 2025 (8h)

He empezado instalando todas las herramientas para poder trabajar.

He hablado con **Ramses** para tener más idea de como plantear el proyecto.

Añado gitignore.

Voy a empezar corriendo el código por primera vez, hay problemas, voy a solucionarlos -> TODO list.

Voy a solucionar problemas 1 a 1, primero, voy a intentar instalar Ollama. -> No puedo.

Voy a trabajar sin el ollama, así que simplemente comentaré la estrategia ollama.

Ahora arreglo los paths, más robusto->no warnings. Había errores con paths absolutos y relativos:

\$ python benchmark/complete_real_comparison.py => Daba error.

Tengo problemas con el import de la estrategia 4. Solucionado.

Solucionado warnings vsc.

Ahora estoy arreglando el formato de los outputs, creando carpetas para cada ejecucion. Solucionado.

Identificar porque los resultados de nuestra estrategia (4) son tan malos. Me ha comentado **Ramses** que me fije en la implementación general pero que empiece por tamaño de chunks, temperatura...

Básicamente hay un gran problema:

Estrategia F1-Score Precision Recall Pred Match Tiempo

1_KIRIs_REAL 0.8000 0.8381 0.7652 105 88 0.0s

2_SNOBERT_REAL 0.3630 0.3072 0.4435 166 51 10.0s

4_TU_RAG_GPT4o 0.0310 0.1429 0.0174 14 2 74.8s

El recall es absurdamente malo. Primero toca entender qué es recall exactamente.

Según nuestro cálculo de métricas (Creo que podríamos añadir más en un futuro para mirar más cuáles son los mejores modelos), tenemos 2 variables:

1. **ground_truth**: Representa todas las anotaciones correctas que deberían ser encontradas.
-> **len(ground_truth)** representa el número total de positivos reales.
2. **exact_matches**: Cuenta cuántas de las predicciones de tu modelo coinciden exactamente.

\$\$

$$\text{recall} = \frac{\text{exact_matches}}{\text{len(ground_truth)}}$$

\$\$

3. **len(predictions)**: Número total de predicciones que hizo el modelo, incluyendo tanto las predicciones correctas como las incorrectas (Falsos Positivos).

\$\$

$$\text{precision} = \frac{\text{exact_matches}}{\text{len(predictions)}}$$

\$\$

Es decir nuestro problema es que la precisión es baja, pero el recall es ridículo. Voy a empezar con esto.

NOTAS 2a reunión del día con **Ramses**:

-> El Snobert no funciona con el modelo que debería, instalarlo en local y ponerlo aquí, en el proyecto (hugging face). Así tendré los mismos resultados que los del README.md. Si ni con eso no es suficiente problema de cortes: e.g. "juancito" lo está dividiendo como "juan" "cito".

TEORÍA:

-BERT los coge casi todos, pero coge algunos que no debería.

-gpt coge algunos pero se deja muchos.

-> Él propone (cree) que la mejor opción será hacer un BERT y luego un pruning con un LLM.

Antes de tratar estrategia 4, solucionaré SNOBERT.

Día 3: 22 Octubre 2025 (4h)

Voy a empezar tratando de descargarme el modelo adecuado de BERT con Hugging Face.

Parece que sí estoy descargando el modelo real...

Antes de seguir voy a hacer una reorganización de programas:

- Cambio nombres de estrategias a 01, 02, 03, 04.

- Cambiar nombre de la comparativa de todos a `all_evaluate_strategies.py`.

- Crear `evaluate_strategies.py`, que se ejecuta con el argumento - para escoger que algoritmo ejecutar.

- Cambio el nombre del directorio `real_strategies` a `strategies`.

Después de todo esto me doy cuenta que puedo hacer una implementación mucho más modular:

Puedo tener solo un fichero `evaluate_strategies.py` y si le entra `strategyID = 0` o simplemente si no se le asigna ningún parámetro (0 por defecto) que ejecute la comparativa general, que simplemente debe ser un for con un vector de las 3 estrategias válidas (01, 02 y 04) + una comparativa final de las métricas F1, recall y precision.

Además, en lugar de estar hard-codeado en el mismo fichero, puede llamar a una clase `Metrics` con funciones como `compare_N_metrics` (N as argument) donde N es el número de estrategias con las que compararé, siempre 1, menos en el caso de `strategyID = 0`, que ahí comparará con N = 3. Por ejemplo ora que sea `print_metrics...`

Vale, re-estructuración hecha. Hago push.

Antes de ponerme a mejorar el gpt 4o, que ya tengo una idea: (primero cachear los 14k embeddings, para solo tener que tardar la 1a vez), voy a:

1. SNOBERT: Arreglar warnings, asegurarme que está loadando correctamente el modelo-> luego arreglar tema corte de palabras.

TODO

Vale, me doy cuenta que **SÍ** está loadando el modelo correcto. Voy a tratar de identificar todas las diferencias entre nuestra implementación de SNOMED y la implementación real:

-Por lo que entiendo básicamente nuestra implementación NO está entrenando el modelo -> Investigar.

TODO

Hago esto ahora:

->Cambiar, nombre de evaluate_strategy.py a main.py

->Cambiar que por defecto no guarde los results solo los printee, se pueda añadir flag para guardarlos.

->README con un poco de informacion como estoy haciendo todo.

Ahora empiezo a mirar como mejorar nuestra implementacion. Problema de que tarda 15minutos (cada vez) a procesar los 14k chunks.

->Cambiar para que el output sque en formato 3 decimales no solo 1 decimal.

Todo esto ya está solucionado.

Ahora he hablado con **Ramses**.

Básicamente el documento conceptos_con_narrativas.csv son 40k lineas y tengo 14k chunks, esto son 3 lineas por chunk.

Hay que probar 1 chunk por linea.

Falta reorganizar ficheros-> build rag index se deberia llamar ontology_preprocessor.py y ponerlo en un directorio /04_strategy.

Además la carpeta assets debería ir dentro de 04_strategy.

Focus en mejorar el rendimiento del GPT (no es muy importante tema API calls).

Faltará volver a ejecutar el builder, ya aprovechar y hacer lo de los chunks.

Además problema de que el fichero ontology.index es 66.5MB y github maximo permite 50 MB, ponerlo en gitignore.

Día 4: 27 Octubre 2025 (8h)

Básiamente he empezado compilando la ontología de nuevo y he aplicado los cambios que **Ramses** decía, a ver que resultados obtengo. He hecho un par de cambios:

Strategy F1-Score Precision Recall Pred Match Time

01_KIRIs 0.8000 0.8381 0.7652 105 88 0.033 s

02_SNOBERT 0.1203 0.4444 0.0696 18 8 1.632 s

04_RAG_GPT 0.0462 0.0690 0.0348 58 4 213.939 s

Las predicciones son muy buenas aunque los match malísimos. Me he dado cuenta que estoy cometiendo un error.

Aún no consigo arreglar el error pero ahora consigo predecir más valores (66), voy a ver.

Problema:

Strategy F1-Score Precision Recall Pred Match Time

01_KIRIs 0.8000 0.8381 0.7652 105 88 0.032 s

02_SNOBERT 0.5593 0.4142 0.8609 239 99 11.651 s

04_RAG_GPT 0.0000 0.0000 0.0000 67 0 112.725 s

Estaba poniendo emojis en la respuesta chatGPT!

Arreglado.

Strategy F1-Score Precision Recall Pred Match Time

01_KIRIs 0.8000 0.8381 0.7652 105 88 0.036 s
04_RAG_GPT 0.4824 0.5714 0.4174 84 48 136.048 s
02_SNOBERT 0.2081 0.3103 0.1565 58 18 3.115 s

Aún mejor, ahora falta mejorar:

- 1-Chunks
- 2-Temperatura
- 3-Prompt
- 4-Embedding

Antes de empezar, voy a organizar mejor el script 04_rag_gpt.py

Me he dado cuenta que la ontología `conceptos_con_narrativas.csv` le faltan conceptos.

He fusionado conceptos:

Strategy F1-Score Precision Recall Pred Match Time

01_KIRIs 0.8000 0.8381 0.7652 105 88 0.032 s
04_RAG_GPT 0.3600 0.4235 0.3130 85 36 129.008 s
02_SNOBERT 0.3346 0.3028 0.3739 142 43 6.946 s

Aún así, faltan conceptos, voy a hacer una ontología híbrida bien hecha...

check_missing_concepts.py

```
=====
=====
=====
```

MISSING CONCEPTS ANALYSIS

Total concepts in training data: 32

Total concepts in ontology: 45440

Missing concepts: 26

Coverage: 18.75%

```
=====
=====
=====
```

MISSING CONCEPT DETAILS (sorted by frequency)

Concept Code Frequency % of Total Annotations

77477000 22 19.13%
55342001 6 5.22%
266257000 6 5.22%
77343006 6 5.22%
230690007 5 4.35%
13791008 4 3.48%
449894001 4 3.48%
230691006 3 2.61%
67889009 3 2.61%
432101006 3 2.61%
25064002 3 2.61%
73211009 2 1.74%
387467008 2 1.74%
433112001 2 1.74%
113091000 2 1.74%
20262006 2 1.74%
38341003 2 1.74%
50582007 2 1.74%
69449002 2 1.74%
422400008 1 0.87%
21454007 1 0.87%
49436004 1 0.87%
87486003 1 0.87%
52674009 1 0.87%
422587007 1 0.87%
8011004 1 0.87%

=====

=====

=====

IMPACT ANALYSIS

Total annotations affected by missing concepts: 88 / 115

Percentage of annotations that cannot be matched: 76.52%

He generado una ontología híbrida con las definiciones de los términos y +2400 conceptos de conceptos_con_narrativas.csv para un total de 2500 conceptos para añadir ruido. He obtenido estos resultados:

Strategy F1-Score Precision Recall Pred Match Time

01_KIRIs 0.8000 0.8381 0.7652 105 88 0.036 s
02_SNOBERT 0.6241 0.3591 2.3826 763 274 31.440 s
04_RAG_GPT 0.3116 0.3690 0.2696 84 31 127.081 s

Es normal que obtenga peores resultados, hay más ruido.

Aún así, lo estaba haciendo mal (conceptos repetidos). Ahora está bien, tengo un script que genera la ontología híbrida: los 32 ground truth concepts y 2468 conceptos más para añadir ruido. He hecho la prueba con una ontología solo con los 32 ground truth concepts y me ha dado 0.6 de F1. Vamos a seguir mejorando el modelo, de momento sin mirar del todo chunks, temperatura, prompt engineering ni embedding, tenemos estos resultados:

Strategy F1-Score Precision Recall Pred Match Time

01_KIRIs 0.8000 0.8381 0.7652 105 88 0.033 s
02_SNOBERT 0.2857 0.2936 0.2783 109 32 4.604 s
04_RAG_GPT 0.2474 0.3038 0.2087 79 24 164.036 s

Refactor hecho, para el próximo día: asegurar que todo funciona, y empezar a mejorar el rag.

Día 5: 30 Octubre 2025 (8h)

Mejoras version 1.1

Después de preparar y analizar todo, me he decidido por aplicar los siguientes cambios:

- 1-Chunks divide a chunks de 3000 chars, ahora probare distintos chunks
- 2-Temperatura a 0 para evitar alucinaciones
- 3-Mejorado el prompt

RESULTADOS - RAG+GPT Strategy

[METRICAS]

Precision: 0.3625

Recall: 0.2522

F1-Score: 0.2974

Coverage: 1.0000

[CONTADORES]

Predicciones: 80

Exact Matches: 0

Partial Matches: 51

Ground Truth: 115

[TIEMPO]

Tiempo de ejecución: 130.98 segundos

Tiempo por nota: 26.20 segundos

Me he dado cuenta de 2 problemas, voy a solucionarlos:

Mejoras version 1.2

El prompt no es bueno y aunque detecte bien la palabra acaba escijiendo el fallback, vo a mejorarlo.
Esto da sobretodo 2 o 3 problemas... voy a ver si los puedo solucionar.

Me he dado cuenta que le estaba pasando demasiado contexto al rag y le estaba añadiendo ruido. Hay muchos problemas que son del embedding, necesita más información con e SAPBERT, ahora lo aplicaré, pero antes voy a inentar mejorarlo aún más.

Vale me he dado cuenta que estaba teniendo problemas y estaba navegando a un camino problematico. Haré commit, trato de arreglarlo, pero sino haré rllback y volvere al score de 0.29.

RESULTADOS - RAG+GPT Strategy

[METRICAS]

Precision: 0.4107

Recall: 0.4000

F1-Score: 0.4053

Coverage: 1.0000

[CONTADORES]

Predicciones: 112

Exact Matches: 0

Partial Matches: 66

Ground Truth: 115

[TIEMPO]

Tiempo de ejecución: 599.83 segundos

Tiempo por nota: 119.97 segundos

=====

🦋 ¡BREAKTHROUGH! F1 = 0.4053!

¡Finalmente superamos el objetivo de 0.3! Cambios ganadores:

- ☑ Threshold más bajo: 1.65 (vs 1.75-1.80 anteriores)
- ☑ Más opciones RAG: k=20 → 12 mostradas (vs 10-8)
- ☑ Prompt agresivo: "Be bold - partial matches acceptable"
- ☑ Fallback codes reducidos: 26/112 (23%) vs 59-78 anteriores (50-70%)
- ☑ Diversidad recuperada: 50 códigos únicos vs 24-34 anteriores

Métricas finales:

Precision: 0.4107

Recall: 0.4000

F1-Score: 0.4053 ← ¡35% mejor que objetivo 0.3!

Partial matches: 66/115

La clave fue equilibrar RAG retrieval (threshold bajo + más opciones) con instrucciones GPT menos conservadoras.

Despues de hacer estos cambios:

Métricas del Óptimo (F1=0.4670):

Precision: 0.4732

Recall: 0.4609

Predicciones: 112

Partial Matches: 59/112 (52.7%)

Códigos Fallback: 26/112 (23.2%)

Códigos Únicos: 41-44

💡 Hallazgos Clave:

Threshold extremadamente sensible: Ventana óptima 1.63-1.65 (± 0.01). Fuera de ese rango, F1 cae >2pp.

k=15 es balance perfecto: k=17 introduce demasiado ruido (-4.69pp).

MAX_DISPLAY=12 óptimo: 14 opciones saturan el contexto de GPT (-1.77pp).

Varianza GPT: Threshold 1.63 y 1.65 dan exactamente F1=0.4670, sugiriendo que GPT tiene ruido inherente.

☒ Configuración óptima confirmada. Los valores están correctos:

Resultado: F1=0.4455 (esta vez bajó ligeramente por varianza de GPT-4o).

```
THRESHOLD = 1.65
TOP_K = 15
MAX_DISPLAY = 12
```

Como viste en los tests anteriores, con estos mismos parámetros obtuvimos:

F1=0.4670 (mejor run)

F1=0.4455 (este run)

Hay una varianza natural de GPT-4o de $\pm 1-2$ puntos porcentuales entre ejecuciones con los mismos parámetros. El rango esperado es F1=0.44-0.47.

Proximo dia

1-Hacer refactor en la nueva pipeline buena (definirla bien tal cual).

2-Comprobar que esta bien hecho el refactor.

3-Empezar a probar cosas nuevas (incluso cambios arquitectura cuando máximo local).

proximo dia

1-seguir mirando lo del NER

2-Mirar recall

3-Mirar ontologia: hacer ontologia sola y luego hacer ontologia sola+conceptos solamente de stroke.

Datos actuales:

RESULTS - KIRIS

Precision: 0.8

Recall: 0.8

F1-Score: 0.8

Coverage: 1.0000

RESULTS - RAG+GPT

Precision: 0.7407

Recall: 0.6250

F1-Score: 0.6780

Coverage: 1.0000

proximo dia

El Diagnóstico Completo

Tu nuevo benchmark SOTA (F1 Estricto) es brillante porque ha expuesto la verdad que el F1 "permisivo" (el de 0.7788) estaba ocultando:

Fallo de NER (Causa tus 92 FN): Tu prompt ner_v5.json (que se enfoca en "diversidad") está haciendo que tu NER no encuentre la mayoría de los 115 spans. Para el benchmark estricto, debes encontrar todas las repeticiones. Tienes que volver a un prompt "agresivo" (como tu ner.json v4.1) que intente encontrar las 115.

Fallo de CODING (Causa tus 39 FP): Cuando tu NER sí encuentra un span (ej: [CT angiography]), tu "Juez" (Etapa 3) le está asignando el concept_id incorrecto. Tu lógica actual de "coger el Top-1 de RAG" es demasiado simple y está fallando.

Es casi seguro que está cogiendo "CT" (77477000) en lugar de "angiography" (77343006).

En el benchmark permisivo, esto contaba como un "acierto" (porque "CT" existía en otra parte).

En el benchmark estricto, esto es un Falso Positivo (para "CT") y un Falso Negativo (para "angiography").

`python scripts/compare_strategies_sota.py` es un buen código ara comparar rag_gpt con kiris:

```
=====
=====
=====
```

COMPARATIVE RESULTS - BENCHMARK SOTA (Strict Evaluation)

Metric RAG-GPT KIRIS Difference

Macro-Average F1 0.2302 0.3798 +0.1496

Micro Precision 0.3279 0.4000 +0.0721

Micro Recall 0.1739 0.3652 +0.1913

Micro F1 0.2273 0.3818 +0.1545

True Positives (TP) 20 42 +22

False Positives (FP) 41 63 +22

False Negatives (FN) 95 73 -22

Execution Time (s) 189.13 0.03 -189.11